Agenda

- AJAX
- Promise
- NodeJs
- User Defined Modules
  - CJS -~~MJS~~
- ~~BuiltIn Modules~~
- ~~Event Loop~~
- ~~http server~~

AJAX

- AJAX stands for Asynchronous JavaScript and XML.
- It is a technique used in web development to update parts of a web page without reloading the entire page.
- By using AJAX, web applications can send and retrieve data asynchronously from a server in the background, improving the user experience by making the app feel faster and more interactive.
- The basic flow of an AJAX request:

1. User Interaction:
   - A user performs an action (e.g., clicking a button).
2. JavaScript Sends a Request:
   - JavaScript sends an HTTP request to the server using the XMLHttpRequest object or the newer fetch API.
3. Server Processes the Request:
   - The server processes the request (e.g., fetches data from a database) and sends back a response.
4. JavaScript Updates the Web Page:
   - JavaScript processes the server's response and updates the web page dynamically, without requiring a page reload.

JSON (JavaScript Object Notation)

- Created by Douglas Crockford is defined as a part of javascript langugage in the early 2000s.
- It wasn't untill 2013 that the format was offically specified
- Javascript Objects are simple associative containers where a string key is mapped with a value.
- It is a lightweight data format used for storing and exchanging data between a server and a web application.
- It is easy to read and write for humans and machines.
- JSON data is written in key-value pairs and uses {} for objects and [] for arrays.
- the data in the form of JSON is converted entirely in the string (using JSON.stringfy()) and then sent through the request or response.

SUNBEAM INFOTECH

- once we receive the request on server or the response on client we convert the string data back into JSON format (using JSON.parse())
- JSON shows up in many different cases.

1. API
2. Configuration Files
3. Log Messages
4. Database storage

```
{
  "name": "John Doe",
  "age": 30,
  "email": "john@example.com",
  "isStudent": false
}

[
  {
    "id": 1,
    "name": "Alice"
  },
  {
    "id": 2,
    "name": "Bob"
  }
]
```

XHR (XMLHttpRequest)

- The original API for making HTTP requests in JavaScript
- It is a JavaScript API that allows developers to make HTTP requests (to fetch data, send data, etc.) from a web server without reloading the page.
- It is commonly used for building AJAX functionality.
- Supported by all browsers (including very old ones)

Steps to Use XHR

1. Create an XHR Object:
    - Use let helper = new XMLHttpRequest();.
2. Configure the Request:
    - Specify the request method (GET, POST, etc.) and the URL
    - helper.open(method, url, async).
    - async:Boolean, true (asynchronous, default) or false (synchronous).
3. Send the Request:
    - helper.send() to send the request.

SUNBEAM INFOTECH

4. Handle the Response:
    - helper.onreadystatechange or helper.onload to process the server's response.

```
const xhr = new XMLHttpRequest();
xhr.open("GET", "https://api.example.com/data", true); // async=true
xhr.onload = function () {
if (xhr.status >= 200 && xhr.status < 300) {
    console.log(JSON.parse(xhr.responseText));
}
else {
    console.error("Request failed:", xhr.statusText);
}
};
xhr.onerror = function () {
console.error("Network error occurred");
};
xhr.send();
```

Promise

- A Promise in JavaScript is an object that represents the eventual completion (or failure) of an asynchronous operation and its resulting value.
- Promises were introduced to solve the lot of callback problem, which occurs when multiple asynchronous operations are nested, leading to messy, hard-to-read, and error-prone code.
- A Promise has three possible states:

1. Pending: The initial state when the Promise is neither fulfilled nor rejected.
2. Fulfilled: The operation completed successfully, and the Promise has a resolved value.
3. Rejected: The operation failed, and the Promise has a reason (error).

- Once a Promise is either fulfilled or rejected, it becomes settled, and its state will not change again.

Reasons for using Promises:

1. Cleaner Code and Readability: Promises provide a structured way to handle asynchronous code, leading to more readable and maintainable code compared to nested callbacks (often referred to as "callback hell" or "pyramid of doom").

```
// Callback-based (less readable)
getData(function(a) {
    processData(a, function(b) {
        displayResult(b, function(c) {
            console.log(c);
        });
    });
});

// Promise-based (more readable and chainable)
```

SUNBEAM INFOTECH

```
getData()
    .then(processData)
    .then(displayResult)
    .then(console.log)
    .catch(handleError);
```

2. Improved Error Handling: Promises offer a centralized and robust mechanism for error handling using the .catch() method. This simplifies error management across a chain of asynchronous operations, unlike callbacks where error handling can become cumbersome and scattered.

3. Better Control over Asynchronous Flow: Promises represent a value that might not be available immediately, allowing you to attach handlers for both successful completion (fulfillment) and failure (rejection) of an asynchronous action. This provides more explicit control over the flow of asynchronous operations.

4. Enabling Async/Await: Promises are the fundamental building block for async/await syntax, which further simplifies asynchronous code by allowing it to be written in a more synchronous-looking style, improving readability and reducing cognitive load.

```
async function fetchDataAndProcess() {
    try {
        const data = await getData();
        const processedData = await processData(data);
        const result = await displayResult(processedData);
        console.log(result);
    } catch (error) {
        console.error("An error occurred:", error);
    }
}
```

SUNBEAM INFOTECH

Fetch API

- It is a modern alternative to XHR.
- Fetch uses Promises, making the code cleaner and easier to manage.
- Easier to read and manage with .then() and .catch() or async/await.
- No need for xhr.open(), xhr.send(), or xhr.onload.
- All configurations are included in the fetch() function.
- Use .catch() to handle errors, such as network issues or HTTP errors.
- Built-in JSON Support: Use .json() to parse JSON responses easily.

```
//using then() and catch()
function btn1Clicked() {
fetch("http://127.0.0.1:5500/data.json", { "method": "GET" })
.then((response) => {
    if (response.ok)
        return response.json()
    else
```

```
        throw new Error("Something went wrong")
})
.then((data) => {
    console.log(data)
})
.catch((error) => {
    console.log("error = " + error)
})
}

//using async/await
async function btn2Clicked() {
try {
    let result = await fetch("http://127.0.0.1:5500/data.json", { "method": "GET"
})
    if (result.ok) {
        const data = await result.json()
        console.log(data)
    }
    else
        throw new Error("Something went wrong")
} catch (error) {
    console.log("error -" + error)
}
}
```

- Both async/await and .then() are ways to handle Promises in JavaScript, but they differ in syntax, readability, and how they handle asynchronous code.
- Which One to Use?

1. Use Async/Await:

- When you have multiple asynchronous tasks that need to be executed in sequence.
- When you want better readability and maintainability.
- When you want to handle errors more cleanly with try/catch.

2. Use .then():

- When you're working with simpler tasks (single or few Promises).
- When you need backward compatibility (older browsers without async/await).

NodeJs

- A software platform that is used to build scalable network applications.
- It is developed by Rayn Dahl in 2009
- It is a javascript runtime built on Google's V8 JavaScript Engine
- Node.js runs the V8 JavaScript engine, the core of Google Chrome, outside of the browser. This allows Node.js to be very performant.
- It provides a set of asynchronous I/O primitives in its standard library that prevent JavaScript code from blocking and generally libraries in Node.js are written using non-blocking paradigms, making blocking behavior the exception rather than the norm.

SUNBEAM INFOTECH

- When an I/O operation like reading from the network, accessing a database or the filesystem, then instead of blocking the thread and wasting CPU cycles waiting, Node.js will resume the operations when the response comes back.
- This allows Node.js to handle thousands of concurrent connections with a single server without introducing the burden of managing thread concurrency, which could be a significant source of bugs.
- It has a unique advantage because millions of frontend developers that write JavaScript for the browser are now able to write the server-side code in addition to the client-side code without the need to learn a completely different language.
- In Node.js the new ECMAScript standards can be used without problems.

Components of NodeJs

1. JavaScript runtime: V8
2. EventLoop : Libuv (A multi-platform support library which focuses on asynchronous I/O, primarily developed for use by Node.js.)
3. Standard Components
    - Filesystem Access
    - Crypto
    - TCP/UDP
    - HTTP
    - Buffer
    - etc

Use Cases of NodeJs

- Where to use
    - I/O bound applications
    - Data Streaming applications
    - IoT applications
    - JSON API based applications
    - Single Page applications
- Where not to use
    - CPU intensive applications
    - Heavy server sided processing

Installation

1. click on the below link to download the node

```
https://nodejs.org/en/download
```

```
# to check for the installed node version
node --version
```

SUNBEAM INFOTECH

```
# to run the node application
node hello.js
```

Module

- It is a collection of functions
- We can share a javascript library with multiple applications with the help of this module
- Their are two types of modules

1. Built-in Modules
    - OS
    - FileSystem
    - HTTP
2. User defined Modules

SUNBEAM INFOTECH